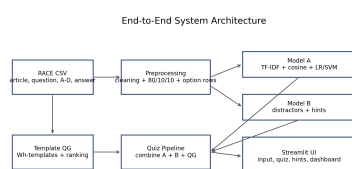


# Intelligent Reading Comprehension and Quiz Generation System

Using Classical Machine Learning, TF-IDF, Cosine Similarity, Template-Based Question Generation, Distractor Generation, and Streamlit

## Artificial Intelligence Project Report BS (CS) Spring 2026



---

|                         |                                                                        |
|-------------------------|------------------------------------------------------------------------|
| <b>Dataset</b>          | RACE - ReAding Comprehension from Examinations                         |
| <b>Primary Approach</b> | Classical ML only: TF-IDF, Logistic Regression, Linear SVM, clustering |
| <b>Interface</b>        | Streamlit web application                                              |
| <b>Report Version</b>   | Final detailed research-style report                                   |

---

# Contents

|          |                                                                            |           |
|----------|----------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Abstract and Executive Summary</b>                                      | <b>1</b>  |
| 1.1      | Abstract . . . . .                                                         | 1         |
| 1.2      | Executive Summary . . . . .                                                | 1         |
| <b>2</b> | <b>Introduction and Problem Definition</b>                                 | <b>2</b>  |
| 2.1      | Motivation . . . . .                                                       | 2         |
| 2.2      | Problem Statement . . . . .                                                | 2         |
| 2.3      | Clarifying What Is Generated . . . . .                                     | 3         |
| 2.4      | Project Objectives . . . . .                                               | 3         |
| <b>3</b> | <b>Related Work and Benchmark Context</b>                                  | <b>4</b>  |
| 3.1      | RACE Dataset . . . . .                                                     | 4         |
| 3.2      | Traditional NLP Representations . . . . .                                  | 4         |
| 3.3      | Transformer-Based Distractor Generation . . . . .                          | 4         |
| 3.4      | Why Neural Benchmarks Are Not Implemented . . . . .                        | 5         |
| <b>4</b> | <b>Dataset, Splitting, and Exploratory Data Analysis</b>                   | <b>5</b>  |
| 4.1      | Dataset Schema . . . . .                                                   | 5         |
| 4.2      | 80/10/10 Split . . . . .                                                   | 6         |
| 4.3      | Option-Level Data Expansion . . . . .                                      | 7         |
| 4.4      | Preprocessing Output . . . . .                                             | 8         |
| <b>5</b> | <b>System Architecture and Software Design</b>                             | <b>8</b>  |
| 5.1      | Layered Architecture . . . . .                                             | 8         |
| 5.2      | Project Structure . . . . .                                                | 9         |
| 5.3      | Artifact Management . . . . .                                              | 10        |
| <b>6</b> | <b>Methodology: Preprocessing, Feature Engineering, and Classical ML</b>   | <b>10</b> |
| 6.1      | Text Cleaning . . . . .                                                    | 10        |
| 6.2      | TF-IDF Representation . . . . .                                            | 10        |
| 6.3      | Article Weighting . . . . .                                                | 11        |
| 6.4      | Cosine Similarity Features . . . . .                                       | 11        |
| 6.5      | Class Imbalance Handling . . . . .                                         | 11        |
| <b>7</b> | <b>Model A: Answer Verification and Template-Based Question Generation</b> | <b>12</b> |
| 7.1      | Model A Task Definition . . . . .                                          | 12        |
| 7.2      | Supervised Models . . . . .                                                | 12        |
| 7.3      | Confusion Matrix Results . . . . .                                         | 13        |
| 7.4      | ROC-AUC and Clustering Baseline . . . . .                                  | 13        |
| 7.5      | Feature Importance . . . . .                                               | 14        |
| 7.6      | Template-Based Question Generation . . . . .                               | 15        |
| 7.7      | Question Generation Evaluation . . . . .                                   | 15        |
| <b>8</b> | <b>Model B: Distractor and Hint Generation</b>                             | <b>16</b> |
| 8.1      | Model B Task Definition . . . . .                                          | 16        |
| 8.2      | Distractor Generation . . . . .                                            | 16        |
| 8.3      | Hint Generation . . . . .                                                  | 17        |
| 8.4      | Model B Evaluation Outputs . . . . .                                       | 17        |
| 8.5      | Comparison to Transformer-Based Distractor Generation . . . . .            | 18        |
| <b>9</b> | <b>Quiz Pipeline and Streamlit User Interface</b>                          | <b>18</b> |
| 9.1      | Quiz Pipeline . . . . .                                                    | 18        |
| 9.2      | Required UI Screens . . . . .                                              | 19        |

|                                                            |           |
|------------------------------------------------------------|-----------|
| 9.3 Session State and Usability . . . . .                  | 23        |
| <b>10 Results and Evaluation</b>                           | <b>24</b> |
| 10.1 Evaluation Metrics . . . . .                          | 24        |
| 10.2 Model A Result Interpretation . . . . .               | 24        |
| 10.3 Question Generation Result Interpretation . . . . .   | 24        |
| 10.4 Model B Result Interpretation . . . . .               | 25        |
| 10.5 Unit Test Results . . . . .                           | 25        |
| 10.6 External Benchmark Comparison . . . . .               | 25        |
| <b>11 Discussion, Limitations, Ethics, and Future Work</b> | <b>26</b> |
| 11.1 Discussion . . . . .                                  | 26        |
| 11.2 Limitations . . . . .                                 | 27        |
| 11.3 Ethical Considerations . . . . .                      | 27        |
| 11.4 Future Work . . . . .                                 | 27        |
| <b>12 Conclusion and Rubric Mapping</b>                    | <b>28</b> |
| 12.1 Conclusion . . . . .                                  | 28        |
| 12.2 Rubric Mapping . . . . .                              | 28        |
| 12.3 Reproducibility Commands . . . . .                    | 29        |

## List of Figures

|    |                                                                                                                                                                                                             |    |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Published neural/transformer RACE-DG benchmark comparison. These values are not produced by our system; they are included as external reference points for transformer-based distractor generation. . . . . | 5  |
| 2  | EDA chart showing the required 80/10/10 dataset split. . . . .                                                                                                                                              | 7  |
| 3  | Class imbalance in option-level answer verification. The 75% negative baseline explains why accuracy alone is misleading. . . . .                                                                           | 8  |
| 4  | System architecture connecting RACE data, preprocessing, Model A, Model B, the quiz pipeline, and Streamlit. . . . .                                                                                        | 9  |
| 5  | Model A comparison across supervised metrics. Linear SVM gives the strongest overall validation performance. . . . .                                                                                        | 12 |
| 6  | Model A confusion matrices. . . . .                                                                                                                                                                         | 13 |
| 7  | ROC-AUC and unsupervised clustering score summary. . . . .                                                                                                                                                  | 14 |
| 8  | Question-generation evaluation metrics from the recorded run. METEOR support exists in the evaluation script but was not numerically logged in the supplied terminal output. . . . .                        | 16 |
| 9  | Model B and integration demonstration counts from supplied logs and tests. . . . .                                                                                                                          | 18 |
| 10 | Streamlit UI - Article Input tab before loading a sample. . . . .                                                                                                                                           | 19 |
| 11 | Streamlit UI - Article Input tab after loading a random RACE sample. . . . .                                                                                                                                | 20 |
| 12 | Streamlit UI - Quiz tab with generated distractors and a checked answer. . . . .                                                                                                                            | 20 |
| 13 | Streamlit UI - Answer checking and Model A feedback message. . . . .                                                                                                                                        | 21 |
| 14 | Streamlit UI - Hint Panel showing progressive hint reveal. . . . .                                                                                                                                          | 21 |
| 15 | Streamlit UI - Developer Dashboard overview with inference latency and current model output. . . . .                                                                                                        | 22 |
| 16 | Streamlit UI - Developer Dashboard chart of Model A option scores. . . . .                                                                                                                                  | 22 |
| 17 | Streamlit UI - Model B distractor and hint quality section. . . . .                                                                                                                                         | 23 |
| 18 | Streamlit UI - Session history and CSV download functionality. . . . .                                                                                                                                      | 23 |

## List of Tables

|    |                                                                                                                                                                                                                                   |    |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | High-level project completion summary. . . . .                                                                                                                                                                                    | 2  |
| 2  | Complete bibliography-oriented summary of the RACE dataset source. . . . .                                                                                                                                                        | 4  |
| 3  | External RACE-DG distractor generation benchmarks from transformer and neural literature. Scores are average BLEU values reported by Taslimipoor et al. . . . .                                                                   | 5  |
| 4  | RACE CSV schema used in the project. . . . .                                                                                                                                                                                      | 6  |
| 5  | Expected 80/10/10 split for the 87,866-row Kaggle CSV shown in the UI run. . . . .                                                                                                                                                | 6  |
| 6  | Example conversion from one MCQ row to four option-level verification samples. . . . .                                                                                                                                            | 7  |
| 7  | Recorded preprocessing output. . . . .                                                                                                                                                                                            | 8  |
| 8  | Important source files and responsibilities. . . . .                                                                                                                                                                              | 9  |
| 9  | Expected model artifacts. . . . .                                                                                                                                                                                                 | 10 |
| 10 | Before/after impact of class imbalance handling. . . . .                                                                                                                                                                          | 11 |
| 11 | Model A supervised validation performance. . . . .                                                                                                                                                                                | 12 |
| 12 | Logistic Regression confusion matrix. Rows are true labels; columns are predicted labels. . . . .                                                                                                                                 | 13 |
| 13 | Linear SVM confusion matrix. Rows are true labels; columns are predicted labels. . . . .                                                                                                                                          | 13 |
| 14 | Model A non-primary evaluation outputs. . . . .                                                                                                                                                                                   | 14 |
| 15 | Important features observed from coefficient analysis. . . . .                                                                                                                                                                    | 14 |
| 16 | Recorded template-based question-generation demo. . . . .                                                                                                                                                                         | 15 |
| 17 | Template question generation evaluation on the recorded sample run. . . . .                                                                                                                                                       | 16 |
| 18 | Recorded Model B inference demo on a simple solar-system example. . . . .                                                                                                                                                         | 17 |
| 19 | Model B and integration test summary from the recorded unit-test output. . . . .                                                                                                                                                  | 17 |
| 20 | Semantic similarity benchmark values from Taslimipoor et al. for generated distractors. Higher gd2d means generated distractors are closer to gold distractors; lower gd2gd means generated distractors are more diverse. . . . . | 18 |
| 21 | UI requirement coverage. . . . .                                                                                                                                                                                                  | 19 |
| 22 | Evaluation metrics used by component. . . . .                                                                                                                                                                                     | 24 |
| 23 | Detailed class-wise Model A report. . . . .                                                                                                                                                                                       | 24 |
| 24 | Question generation output analysis. . . . .                                                                                                                                                                                      | 25 |
| 25 | Unit test completion summary. . . . .                                                                                                                                                                                             | 25 |
| 26 | Classical implementation versus external neural benchmark context. . . . .                                                                                                                                                        | 26 |
| 27 | Rubric coverage summary. . . . .                                                                                                                                                                                                  | 28 |

# 1 Abstract and Executive Summary

## 1.1 Abstract

This report presents the design, implementation, evaluation, and user-interface integration of an Intelligent Reading Comprehension and Quiz Generation System built on the RACE dataset. The system follows the project constraint of using traditional machine learning rather than implementing neural networks as the core model. It combines a supervised answer-verification component, a template-based question-generation component, a distractor and hint generation component, and a Streamlit front end. Model A performs option-level answer verification using TF-IDF text representations, cosine-similarity features, Logistic Regression, and Linear SVM. Model A also includes classical template-based question generation using rule-based Wh-question templates and lexical ranking. Model B generates distractors and progressive hints through TF-IDF, candidate extraction, semantic similarity, and rule-based ranking. The final application allows a user to load RACE passages, answer MCQs, reveal hints, inspect model confidence, and view a developer dashboard.

The best supervised Model A result recorded in the project logs is achieved by the Linear SVM, with accuracy 0.6055 and macro F1 0.5608 on the validation sample. Logistic Regression achieved accuracy 0.5745 and macro F1 0.5230 after class-weight balancing and hyperparameter tuning. Template-based question generation achieved BLEU 0.016998 and ROUGE-L 0.132267 on a sample run against RACE reference questions. These values are low but expected for non-neural rule-based generation because RACE questions are often paraphrased and reasoning-oriented. External transformer-based distractor-generation benchmarks from Taslimipoor et al. report stronger BLEU results on RACE-DG, but these models use neural T5/GPT approaches and are included only as a comparison reference, not as part of the implemented system.

## 1.2 Executive Summary

The system is best understood as a classical NLP pipeline rather than a transformer system. Its main strengths are reproducibility, interpretability, modularity, and compliance with the traditional ML requirement. The implementation includes the complete workflow expected by the project: data loading and splitting, preprocessing, supervised answer verification, unsupervised clustering, template question generation, distractor and hint generation, evaluation metrics, unit tests, and a working Streamlit interface. The UI includes all required screens: Article Input, Quiz, Hint Panel, and Developer Dashboard.

Table 1: High-level project completion summary.

| Project Component        | Implemented Capability                                                                              | Status                |
|--------------------------|-----------------------------------------------------------------------------------------------------|-----------------------|
| Data pipeline            | Single Kaggle CSV split into train/dev/test using 80/10/10;                                         | Complete              |
| Model A                  | option-level verification rows generated<br>TF-IDF + cosine + Logistic Regression +                 | Complete              |
| Model B                  | Linear SVM + KMeans + template QG<br>Distractor generation and hint generation                      | Complete / polishable |
| Quiz pipeline            | using classical NLP logic<br>Integrates Model A, Model B, RACE                                      | Complete              |
| UI                       | samples, generated distractors, hints, and<br>UI state<br>Streamlit application with Article Input, | Complete              |
| Evaluation               | Quiz, Hints, and Developer Dashboard<br>Accuracy, F1, precision, recall, confusion                  | Complete              |
| Report and documentation | matrices, BLEU, ROUGE-L, tests,<br>published comparison<br>Detailed research-style report, code     | This document         |
|                          | snippets, screenshots, benchmark<br>comparison                                                      |                       |

*Note: Results are taken from the team’s recorded local execution logs, generated CSV outputs, supplied screenshots, and cited benchmark literature.*

## 2 Introduction and Problem Definition

### 2.1 Motivation

Multiple-choice reading comprehension exercises are widely used in schools and standardized tests because they evaluate reading, inference, vocabulary, and reasoning ability. However, constructing such exercises manually is time-consuming. A complete intelligent quiz system must support several tasks at once: it must understand a passage, judge which answer option is correct, create plausible but incorrect distractors, provide hints without revealing the answer too early, and present the complete workflow through an accessible interface.

The RACE dataset provides a strong foundation for this project because it contains English reading passages and human-written multiple-choice questions taken from middle-school and high-school English examinations. The original RACE paper describes the dataset as containing nearly 28,000 passages and nearly 100,000 questions, designed to test reading comprehension and reasoning [1]. The project implementation uses the Kaggle-packaged RACE CSV format with columns for article, question, four answer options, and the correct answer label.

### 2.2 Problem Statement

The project objective is to build an Intelligent Reading Comprehension and Quiz Generation System using classical ML. The system must process a passage and question, verify correct answers, generate or rank distractors, generate hints, and expose the complete workflow through a user interface. The central technical challenge is that text must be converted into numerical representations before ML

models can process it. This is handled using TF-IDF vectorization, cosine similarity, and handcrafted lexical features.

## 2.3 Clarifying What Is Generated

A major source of confusion in this project is the phrase “quiz generation.” The final system includes several levels of generation and retrieval:

- **RACE-based quiz delivery:** the system can load existing RACE articles, questions, answer options, and gold labels.
- **Answer verification:** Model A predicts which option is correct, even though the dataset contains the gold label for supervised training and evaluation.
- **Distractor generation:** Model B can replace or supplement wrong answer choices using generated distractor candidates.
- **Hint generation:** Model B ranks relevant passage sentences and presents progressive hints.
- **Template question generation:** Model A-QG can generate new Wh-style questions from article sentences using traditional rule-based methods.

Thus, the system is not merely a dataset viewer. It is a hybrid classical NLP system that reuses RACE as the benchmark while generating enhanced quiz elements.

## 2.4 Project Objectives

The implementation targets the following objectives:

1. Load and preprocess the RACE dataset.
2. Convert MCQ rows into option-level binary classification samples.
3. Train at least two traditional ML models for answer verification.
4. Use TF-IDF and cosine similarity as core text features.
5. Address class imbalance through class weighting.
6. Add unsupervised or semi-supervised analysis through clustering.
7. Generate template-based questions using traditional NLP.
8. Generate distractors and hints using classical ranking and similarity methods.
9. Build a usable Streamlit interface with all required screens.
10. Compare classical results with reported neural/transformer benchmarks.



### 3 Related Work and Benchmark Context

#### 3.1 RACE Dataset

RACE was introduced by Lai et al. as a large-scale reading comprehension dataset collected from English examinations for Chinese middle-school and high-school students [1]. Its questions are written by human experts and often require reasoning rather than shallow lexical matching. This makes the dataset more challenging than many earlier comprehension benchmarks. The dataset structure is especially suitable for MCQ systems because each item includes one passage, one question, four answer choices, and one correct answer label.

Table 2: Complete bibliography-oriented summary of the RACE dataset source.

| Field               | Bibliographic / Dataset Detail                                                                  |
|---------------------|-------------------------------------------------------------------------------------------------|
| Dataset name        | RACE: Large-scale ReAding Comprehension Dataset From Examinations                               |
| Authors             | Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy                                |
| Venue               | Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing          |
| Pages               | 785–794                                                                                         |
| Location            | Copenhagen, Denmark                                                                             |
| Publisher           | Association for Computational Linguistics                                                       |
| DOI                 | 10.18653/v1/D17-1082                                                                            |
| Dataset description | English reading comprehension passages and MCQs from middle-school and high-school examinations |
| Kaggle package used | Ankit Dhiman, RACE Dataset, Kaggle; used as CSV packaging for the project implementation        |
| Citation key        | lai2017race                                                                                     |

#### 3.2 Traditional NLP Representations

Before transformer models became dominant, many NLP systems represented documents using sparse lexical features. TF-IDF remains a strong baseline because it is simple, interpretable, memory-efficient, and works well with linear classifiers. The project uses TF-IDF as the primary feature representation and combines it with cosine similarity to model relationships between article, question, and answer option text.

#### 3.3 Transformer-Based Distractor Generation

Taslimipoor et al. propose a two-step distractor-generation approach using transformer-based models. Their system first generates answer options and then discriminates correct from incorrect options, using clustering to improve distractor diversity [2]. This paper is important for our project because it gives published neural/transformer results on RACE-DG, including T5 and GPT baselines, without requiring us to implement neural networks ourselves.

Table 3: External RACE-DG distractor generation benchmarks from transformer and neural literature. Scores are average BLEU values reported by Taslimipoor et al.

| System             | BLEU1 | BLEU2 | BLEU3 | BLEU4 |
|--------------------|-------|-------|-------|-------|
| HSA [8]            | 0.28  | 0.14  | 0.08  | 0.05  |
| EDGE [9]           | 0.32  | 0.17  | 0.10  | 0.07  |
| Baseline T5 [2]    | 0.20  | 0.13  | 0.09  | 0.07  |
| Zero-shot GPT [10] | 0.23  | 0.12  | 0.06  | 0.04  |
| One-shot GPT [10]  | 0.27  | 0.15  | 0.09  | 0.05  |
| Two-step DG [2]    | 0.29  | 0.18  | 0.13  | 0.10  |

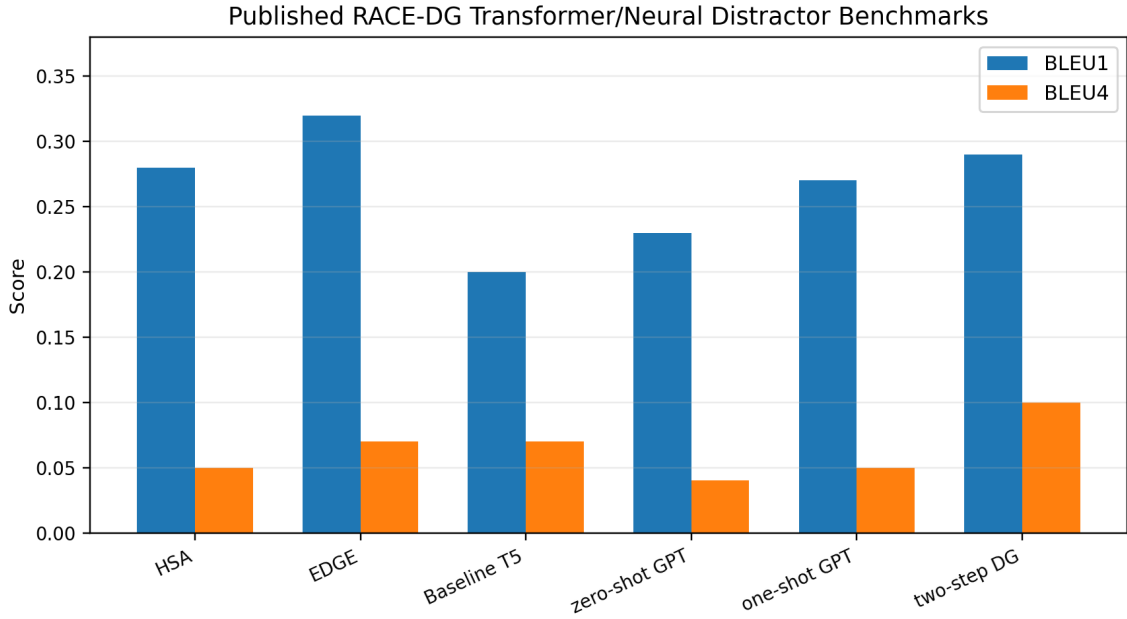


Figure 1: Published neural/transformer RACE-DG benchmark comparison. These values are not produced by our system; they are included as external reference points for transformer-based distractor generation.

### 3.4 Why Neural Benchmarks Are Not Implemented

The project constraint clarified by the teaching staff is that the implementation should use traditional ML models specifically. Therefore, BERT/T5 are used as comparison benchmarks from published work rather than being implemented as the project core. This avoids violating the traditional-ML requirement while still satisfying the instruction to check classical results against stronger neural approaches.

## 4 Dataset, Splitting, and Exploratory Data Analysis

### 4.1 Dataset Schema

The RACE CSV schema used by the project contains the columns shown in Table 4. Each row corresponds to one MCQ item.

Table 4: RACE CSV schema used in the project.

| Column   | Type   | Used by                         | Description                                     |
|----------|--------|---------------------------------|-------------------------------------------------|
| id       | String | Data reference                  | Unique identifier for the passage/question item |
| article  | String | Model A, Model B, UI            | Full reading passage                            |
| question | String | Model A, Model B, QG evaluation | MCQ question associated with the passage        |
| A        | String | Model A, UI                     | Option A text                                   |
| B        | String | Model A, UI                     | Option B text                                   |
| C        | String | Model A, UI                     | Option C text                                   |
| D        | String | Model A, UI                     | Option D text                                   |
| answer   | String | Training/evaluation             | Gold answer label: A, B, C, or D                |

## 4.2 80/10/10 Split

The teaching staff instructed that the Kaggle dataset should be loaded as a single file and split manually into 80% training, 10% validation/dev, and 10% test. The implementation adds `src/split_dataset.py`, which detects or accepts a source CSV and produces `train.csv`, `dev.csv`, and `test.csv`. The split uses a reproducible random seed.

```

1 train_df, temp_df = train_test_split(
2     df, test_size=0.20, random_state=random_state, shuffle=True
3 )
4 dev_df, test_df = train_test_split(
5     temp_df, test_size=0.50, random_state=random_state, shuffle=True
6 )
7
8 train_df.to_csv(RAW_DIR / "train.csv", index=False)
9 dev_df.to_csv(RAW_DIR / "dev.csv", index=False)
10 test_df.to_csv(RAW_DIR / "test.csv", index=False)

```

Listing 1: Core logic from the 80/10/10 split script.

Table 5: Expected 80/10/10 split for the 87,866-row Kaggle CSV shown in the UI run.

| Split          | Proportion | Approx. rows | Purpose                                           |
|----------------|------------|--------------|---------------------------------------------------|
| Train          | 80%        | 70,292       | Fitting TF-IDF, LR, SVM, Model B vectorizers      |
| Dev/Validation | 10%        | 8,787        | Model selection and evaluation during development |
| Test           | 10%        | 8,787        | Final hold-out evaluation                         |

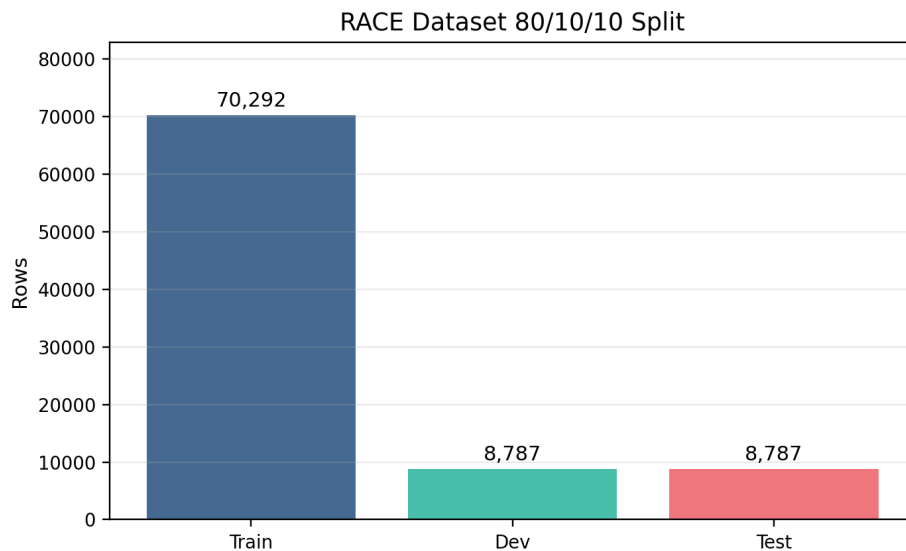


Figure 2: EDA chart showing the required 80/10/10 dataset split.

### 4.3 Option-Level Data Expansion

The project requirement specifies option-level training. Each original MCQ row is expanded into four binary samples. The correct option receives label 1 and the other three options receive label 0. This creates an unavoidable 3:1 class imbalance.

Table 6: Example conversion from one MCQ row to four option-level verification samples.

| Option | Input text to model           | Label              |
|--------|-------------------------------|--------------------|
| A      | article + question + option A | 0 if A is not gold |
| B      | article + question + option B | 0 if B is not gold |
| C      | article + question + option C | 1 if C is gold     |
| D      | article + question + option D | 0 if D is not gold |

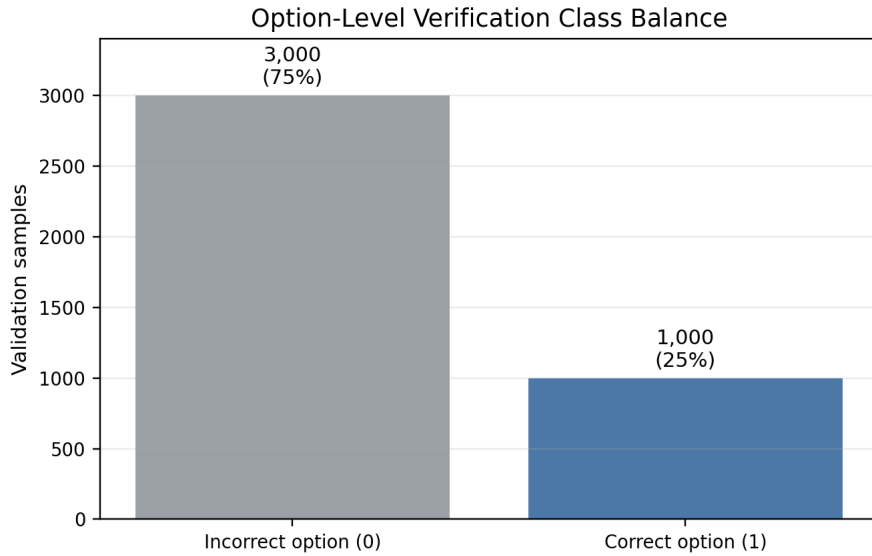


Figure 3: Class imbalance in option-level answer verification. The 75% negative baseline explains why accuracy alone is misleading.

#### 4.4 Preprocessing Output

The recorded preprocessing run confirmed that rows were expanded into option-level training samples. A 5,000-row training sample became 20,000 binary examples, and 1,000-row dev/test samples became 4,000 binary examples each.

Table 7: Recorded preprocessing output.

| Dataset stage | Original rows | Processed option-level rows |
|---------------|---------------|-----------------------------|
| Train sample  | 5,000         | 20,000                      |
| Dev sample    | 1,000         | 4,000                       |
| Test sample   | 1,000         | 4,000                       |

## 5 System Architecture and Software Design

### 5.1 Layered Architecture

The implementation follows a three-layer architecture: data layer, model layer, and UI layer. The data layer handles CSV loading, splitting, preprocessing, and option-level conversion. The model layer includes Model A and Model B. The UI layer uses Streamlit to expose the system to the user.

## End-to-End System Architecture

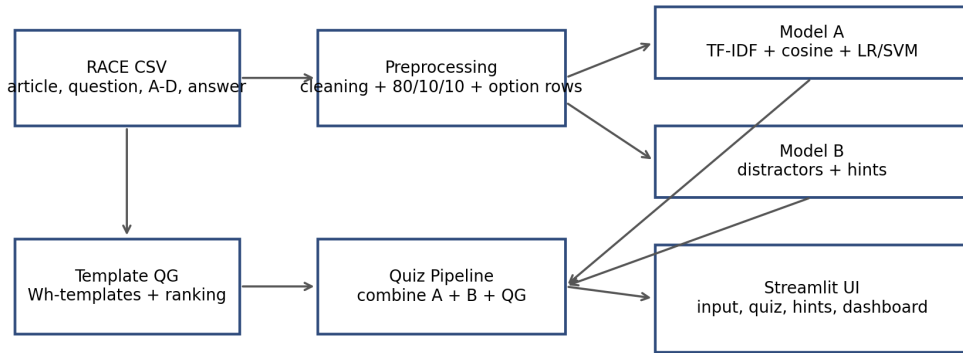


Figure 4: System architecture connecting RACE data, preprocessing, Model A, Model B, the quiz pipeline, and Streamlit.

## 5.2 Project Structure

The branch is organized into clearly separated modules. This improves maintainability and makes it easy to run or replace individual components.

Table 8: Important source files and responsibilities.

| File                                         | Responsibility                                                                                     |
|----------------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>src/split_dataset.py</code>            | Splits one Kaggle CSV into train/dev/test using 80/10/10                                           |
| <code>src/preprocessing.py</code>            | Cleans text and converts MCQ rows into option-level binary samples                                 |
| <code>src/model_a_train.py</code>            | Trains Logistic Regression, Linear SVM, TF-IDF vectorizer, KMeans and supporting metrics           |
| <code>src/inference.py</code>                | Loads Model A artifacts and predicts the best answer option                                        |
| <code>src/model_a_qg.py</code>               | Generates template-based questions using classical NLP methods                                     |
| <code>src/model_b_train.py</code>            | Builds Model B vectorizer/ranker artifacts                                                         |
| <code>src/model_b_inference.py</code>        | Generates distractors and hints                                                                    |
| <code>src/quiz_pipeline.py</code>            | Combines Model A, Model B, generated questions, and RACE samples into quiz items                   |
| <code>src/question_generation_eval.py</code> | Evaluates generated questions against RACE references using BLEU, ROUGE-L, and METEOR if available |
| <code>ui/app.py</code>                       | Streamlit interface                                                                                |
| <code>tests/test_inference.py</code>         | Unit tests for inference, evaluation, and Model B functions                                        |

### 5.3 Artifact Management

The system saves serialized sklearn objects using `joblib`. The file extension may be `.pkl` or `.joblib`; the key point is that `joblib.dump()` is used. This is appropriate for sklearn models and TF-IDF vectorizers because they may contain sparse matrices and large vocabularies.

Table 9: Expected model artifacts.

| Artifact                                                        | Purpose                                    |
|-----------------------------------------------------------------|--------------------------------------------|
| <code>models/model_a/traditional/logistic_regression.pkl</code> | Balanced Logistic Regression verifier      |
| <code>models/model_a/traditional/linear_svm.pkl</code>          | Balanced Linear SVM verifier               |
| <code>models/model_a/traditional/tfidf_vectorizer.pkl</code>    | Fitted TF-IDF vocabulary used at inference |
| <code>models/model_a/traditional/kmeans.pkl</code>              | Unsupervised clustering baseline           |
| <code>models/model_a/question_generation_eval.csv</code>        | Question-generation evaluation results     |
| <code>models/model_b/traditional/model_b_vectorizer.pkl</code>  | Model B TF-IDF vectorizer                  |

## 6 Methodology: Preprocessing, Feature Engineering, and Classical ML

### 6.1 Text Cleaning

Text preprocessing normalizes casing, removes punctuation where appropriate, strips extra whitespace, and prepares concatenated text fields. This is necessary because sparse lexical models treat surface forms literally. Uncontrolled punctuation or casing can inflate the vocabulary and reduce feature reuse.

### 6.2 TF-IDF Representation

TF-IDF represents each document as a sparse vector weighted by term importance. A word receives a high score when it appears frequently in a document but is relatively rare across the corpus. The implementation uses the recommended settings: English stopword removal, sublinear term frequency, unigrams and bigrams, and a capped vocabulary.

```

1 vectorizer = TfidfVectorizer(
2     max_features=15000,
3     stop_words="english",
4     sublinear_tf=True,
5     ngram_range=(1, 2),
6     min_df=2,
7     max_df=0.95
8 )

```

Listing 2: TF-IDF vectorizer settings used in Model A.

### 6.3 Article Weighting

The combined text gives the article extra weight by repeating it. This helps the model prioritize passage evidence when classifying an answer option.

```
1 combined_text = article + " " + article + " " + question + " " + option_text
```

Listing 3: Combined text used for option-level answer verification.

### 6.4 Cosine Similarity Features

In addition to TF-IDF vectors, the system computes three numerical similarity features:

1. question-option similarity,
2. article-option similarity,
3. article-question similarity.

The coefficient analysis showed that article-option cosine similarity is the strongest positive feature for both Logistic Regression and Linear SVM. This is intuitive: if an option is lexically/semantically closer to the article, it is more likely to be the correct answer.

```
1 q_opt = cosine_similarity(question_vectors, option_vectors).diagonal()
2 art_opt = cosine_similarity(article_vectors, option_vectors).diagonal()
3 art_q = cosine_similarity(article_vectors, question_vectors).diagonal()
4 cosine_features = np.vstack([q_opt, art_opt, art_q]).T
5 X_train = hstack([X_train_tfidf, train_cosine])
```

Listing 4: Cosine similarity feature construction.

### 6.5 Class Imbalance Handling

Option-level training produces 75% incorrect samples and 25% correct samples. Early training produced 0.7500 accuracy but F1 of 0.0000 because the model predicted only class 0. This was fixed using class weighting.

```
1 logistic_model = LogisticRegression(max_iter=1000, class_weight="balanced")
2 svm_model = LinearSVC(class_weight="balanced")
```

Listing 5: Balanced classifiers used to avoid the fake 75% baseline.

Table 10: Before/after impact of class imbalance handling.

| Stage                 | Accuracy | Positive-class F1 | Interpretation                                          |
|-----------------------|----------|-------------------|---------------------------------------------------------|
| Before balancing      | 0.7500   | 0.0000            | Degenerate classifier predicts all options as incorrect |
| After balancing - LR  | 0.5745   | 0.37              | Lower accuracy but meaningful correct-option detection  |
| After balancing - SVM | 0.6055   | 0.42              | Best balanced supervised result                         |



## 7 Model A: Answer Verification and Template-Based Question Generation

### 7.1 Model A Task Definition

Model A is the answer-verification engine. It receives an article, a question, and one option, then predicts whether that option is correct. During four-option inference, the same model scores A, B, C, and D, then selects the highest-scoring option.

$$\text{Model A}(\text{article}, \text{question}, \text{option}) \rightarrow P(\text{option} = \text{correct}) \quad (1)$$

### 7.2 Supervised Models

The final supervised Model A includes Logistic Regression and Linear SVM. Both models are trained on the same TF-IDF + cosine feature matrix. Logistic Regression provides probability estimates through `predict_proba`, while Linear SVM provides strong sparse-text classification performance through margin-based separation.

Table 11: Model A supervised validation performance.

| Model               | Accuracy | Macro F1 | Macro Precision | Macro Recall | ROC-AUC |
|---------------------|----------|----------|-----------------|--------------|---------|
| Logistic Regression | 0.5745   | 0.5230   | 0.54            | 0.55         | 0.5629  |
| Linear SVM          | 0.6055   | 0.5608   | 0.57            | 0.59         | N/A     |

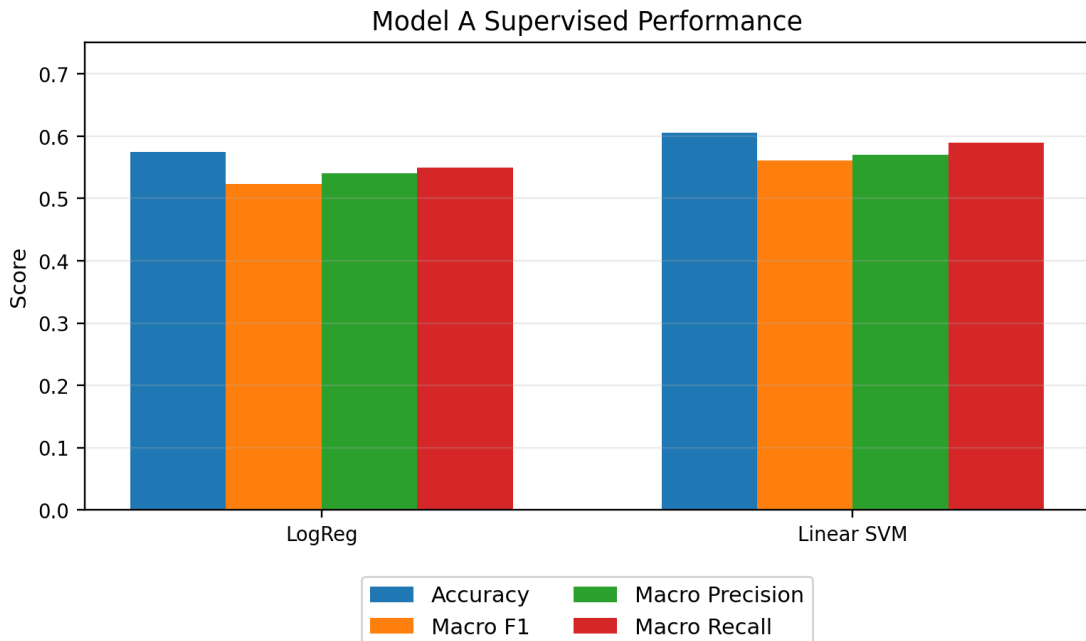


Figure 5: Model A comparison across supervised metrics. Linear SVM gives the strongest overall validation performance.

### 7.3 Confusion Matrix Results

The confusion matrices show that the balanced models predict both classes. This is a major improvement over the original degenerate 75% baseline.

Table 12: Logistic Regression confusion matrix. Rows are true labels; columns are predicted labels.

|        | Predicted 0 | Predicted 1 |
|--------|-------------|-------------|
| True 0 | 1806        | 1194        |
| True 1 | 508         | 492         |

Table 13: Linear SVM confusion matrix. Rows are true labels; columns are predicted labels.

|        | Predicted 0 | Predicted 1 |
|--------|-------------|-------------|
| True 0 | 1849        | 1151        |
| True 1 | 427         | 573         |

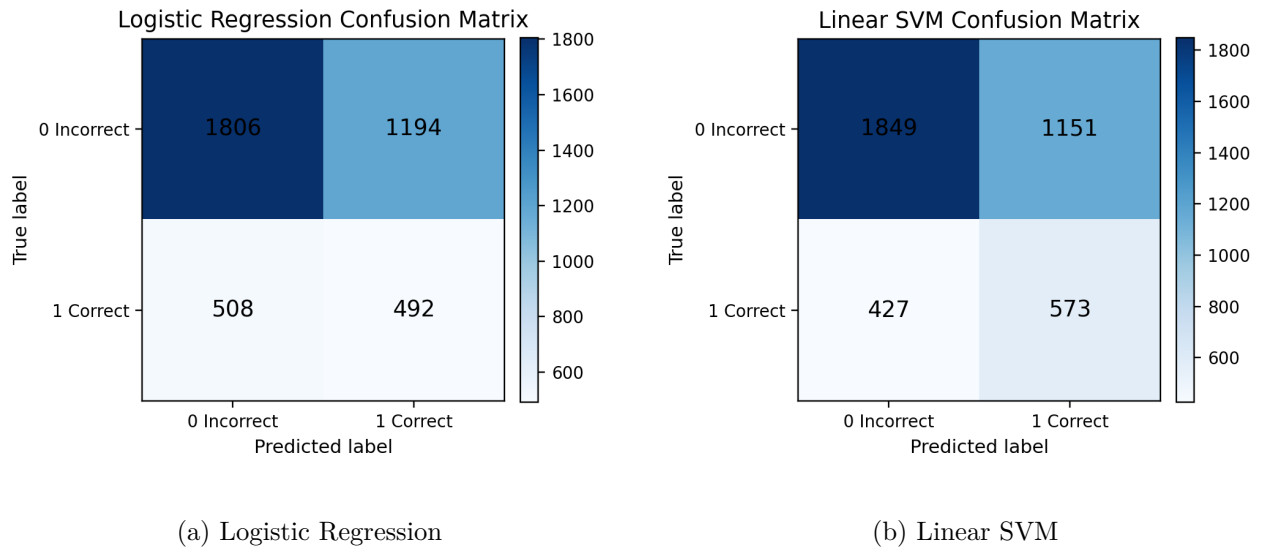


Figure 6: Model A confusion matrices.

### 7.4 ROC-AUC and Clustering Baseline

Logistic Regression produced ROC-AUC 0.5629. KMeans clustering was also implemented to satisfy the unsupervised exploration requirement. The KMeans silhouette score is 0.0037, which is low but expected for sparse high-dimensional text data where clusters are not naturally separable.

Table 14: Model A non-primary evaluation outputs.

| Component                     | Score  | Interpretation                                                    |
|-------------------------------|--------|-------------------------------------------------------------------|
| Logistic Regression ROC-AUC   | 0.5629 | Slightly above random ranking; useful for probability analysis    |
| KMeans silhouette score       | 0.0037 | Weak unsupervised separation, expected for sparse NLP vectors     |
| GridSearchCV best C           | 0.1    | Stronger regularization selected for balanced Logistic Regression |
| GridSearchCV best CV macro F1 | 0.5169 | Cross-validated estimate during tuning                            |

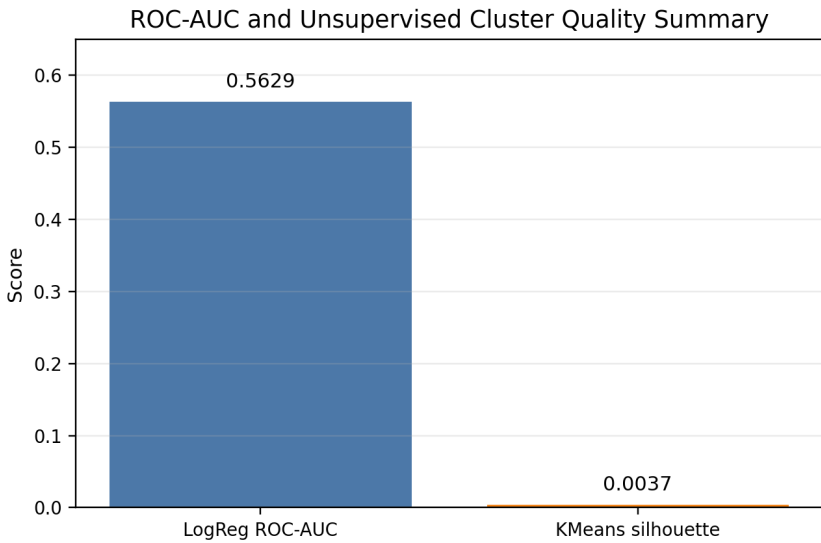


Figure 7: ROC-AUC and unsupervised clustering score summary.

7.5 Feature Importance

The strongest positive features included `cosine_article_option` for both supervised models. This confirms the feature design: answer options closer to the article are more likely to be correct. Negative features included common generic question patterns such as “following true” and “passage”, which often correspond to difficult global questions where lexical overlap alone is insufficient.

Table 15: Important features observed from coefficient analysis.

| Model | Direction | Example feature                         | Interpretation                                                          |
|-------|-----------|-----------------------------------------|-------------------------------------------------------------------------|
| LR    | Positive  | <code>cosine_article_option</code>      | Correct answers often align strongly with article wording               |
| LR    | Positive  | <code>life, attitude, experience</code> | Content terms associated with correct options in sampled data           |
| LR    | Negative  | <code>cosine_question_option</code>     | High question-option overlap can be misleading for some question types  |
| SVM   | Positive  | <code>cosine_article_option</code>      | Most important engineered signal                                        |
| SVM   | Negative  | <code>didnt like, following true</code> | Dataset-specific lexical patterns associated with incorrect predictions |

## 7.6 Template-Based Question Generation

The project now includes classical question generation in `model_a_qg.py`. It does not use BERT, T5, or any neural model. Instead, it identifies important sentences, extracts candidate spans, and applies Wh-question templates such as *Who*, *When*, *Where*, *Why*, and *What did*. The following recorded output demonstrates the component.

Table 16: Recorded template-based question-generation demo.

| Rank | Generated question                                           | Answer                                          | Template    |
|------|--------------------------------------------------------------|-------------------------------------------------|-------------|
| 1    | Who discovered radium in 1898 while working in Paris?        | Marie Curie                                     | who-subject |
| 2    | When did Marie Curie discover radium while working in Paris? | 1898                                            | when        |
| 3    | Where did Marie Curie discover radium in 1898 while working? | in Paris                                        | where       |
| 4    | Why did Curie’s research become important?                   | it helped doctors develop new cancer treatments | why         |
| 5    | What did Marie Curie discover?                               | radium in 1898 while working in Paris           | what-action |

```

1 def generate_questions(article, top_k=5):
2     sentences = split_sentences(article)
3     scored_sentences = score_sentences_with_tfidf(sentences, article)
4     candidates = []
5     for sentence in scored_sentences:
6         candidates.extend(generate_question_candidates_from_sentence(sentence))
7     ranked = rank_generated_questions(article, candidates)
8     return ranked[:top_k]
```

Listing 6: Simplified template-based question generation flow.

## 7.7 Question Generation Evaluation

Generated questions are compared against RACE reference questions using overlap-based metrics. BLEU measures n-gram overlap, ROUGE-L measures longest-common-subsequence overlap, and METEOR is supported in the script when NLTK resources are available. The supplied run logged BLEU and ROUGE-L values.

Table 17: Template question generation evaluation on the recorded sample run.

| Metric  | Mean score                             | Interpretation                                                                |
|---------|----------------------------------------|-------------------------------------------------------------------------------|
| BLEU    | 0.016998                               | Very low exact n-gram overlap; expected for rule-based QG                     |
| ROUGE-L | 0.132267                               | Some lexical/sequence overlap with reference RACE questions                   |
| METEOR  | Not logged in supplied terminal output | Supported by latest evaluation script; rerun required for exact numeric value |

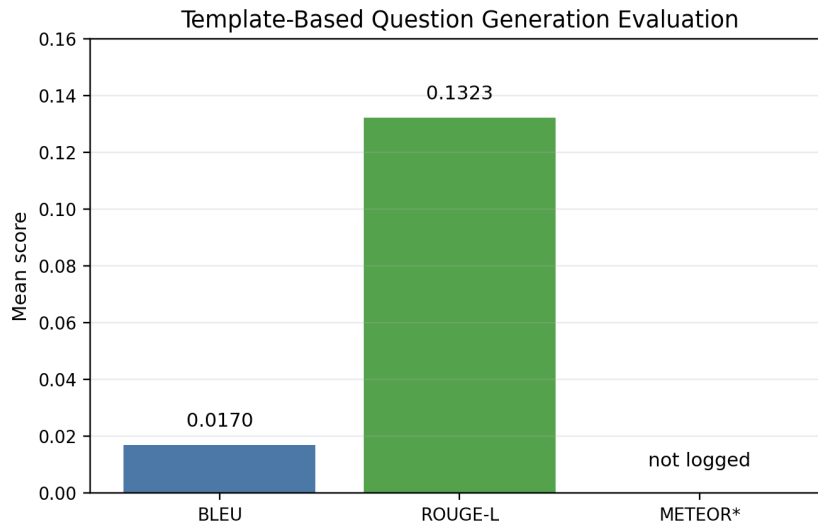


Figure 8: Question-generation evaluation metrics from the recorded run. METEOR support exists in the evaluation script but was not numerically logged in the supplied terminal output.

## 8 Model B: Distractor and Hint Generation

### 8.1 Model B Task Definition

Model B receives a passage, question, and correct answer, then generates plausible distractors and hints. In the final quiz interface, Model B can replace original RACE distractors with generated distractors. It also provides progressive hints ranked from general to more specific.

$$\text{Model B}(\text{article}, \text{question}, \text{correct answer}) \rightarrow \{\text{distractors}, \text{hints}\} \quad (2)$$

### 8.2 Distractor Generation

The system uses candidate extraction, filtering, similarity scoring, and ranking. The aim is to generate distractors that are not identical to the correct answer, are relevant to the article/question, and have enough plausibility to function as wrong options.

Table 18: Recorded Model B inference demo on a simple solar-system example.

| Output type  | Generated output                                       |
|--------------|--------------------------------------------------------|
| Distractor 1 | The Earth                                              |
| Distractor 2 | The Moon                                               |
| Distractor 3 | Mars                                                   |
| Hint 1       | Mars and Jupiter are also planets in the solar system. |
| Hint 2       | The Earth revolves around the Sun once every year.     |
| Hint 3       | The Moon revolves around the Earth.                    |

### 8.3 Hint Generation

Hints are generated by scoring passage sentences against the question. The UI reveals hints one at a time. This is pedagogically better than revealing the answer immediately because the user is gradually guided toward the relevant part of the passage.

```

1 question_vec = vectorizer.transform([clean_text(question)])
2 sentence_vecs = vectorizer.transform([clean_text(s) for s in sentences])
3 similarities = cosine_similarity(question_vec, sentence_vecs)[0]
4 ranked = sorted(zip(sentences, similarities), key=lambda x: x[1])
5 hints = [sentence for sentence, score in ranked[-top_k:]]

```

Listing 7: Simplified hint generation logic.

### 8.4 Model B Evaluation Outputs

Unit tests confirm that Model B functions return strings, generate the requested number of distractors, avoid returning the exact correct answer, handle empty inputs, and generate hints. The supplied unit-test suite includes both mocked and real Model B inference tests.

Table 19: Model B and integration test summary from the recorded unit-test output.

| Test group                            | Checks covered                                                           | Status |
|---------------------------------------|--------------------------------------------------------------------------|--------|
| Mocked Model B inference              | cleaning, stopwords, fallback, uniqueness, hints                         | Passed |
| Real Model B inference                | distractor count, distractor type, difference from correct answer, hints | Passed |
| Model B distractor evaluation utility | all-valid and invalid-fallback cases                                     | Passed |
| Full unit test run                    | 40 tests in 5.632 seconds                                                | OK     |

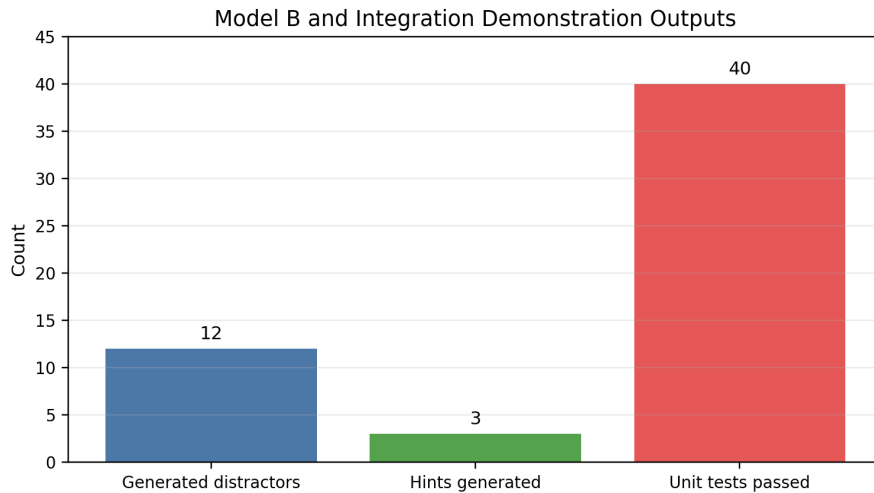


Figure 9: Model B and integration demonstration counts from supplied logs and tests.

## 8.5 Comparison to Transformer-Based Distractor Generation

The transformer-based paper by Taslimipoor et al. shows that neural generation models can achieve stronger BLEU results on RACE-DG than simple rule-based systems. This is expected because T5/GPT models contain contextual representations learned from large-scale pretraining. However, this project intentionally stays within classical ML constraints. The neural benchmark is therefore used as a reference point, not as a target to reproduce.

Table 20: Semantic similarity benchmark values from Taslimipoor et al. for generated distractors. Higher gd2d means generated distractors are closer to gold distractors; lower gd2gd means generated distractors are more diverse.

| System        | gd2c  | gd2d d1 | gd2d d2 | gd2d d3 | gd2gd |
|---------------|-------|---------|---------|---------|-------|
| Gold          | 46.79 | –       | –       | –       | 33.80 |
| Baseline T5   | 49.25 | 56.26   | 37.90   | 17.73   | 58.55 |
| Zero-shot GPT | 42.21 | 48.06   | 46.79   | 46.29   | 51.99 |
| One-shot GPT  | 43.00 | 50.07   | 48.81   | 47.49   | 49.87 |
| Two-step DG   | 42.28 | 54.00   | 51.84   | 49.39   | 43.56 |

## 9 Quiz Pipeline and Streamlit User Interface

### 9.1 Quiz Pipeline

The quiz pipeline combines the output of Model A and Model B into a single data object used by the UI. It loads a RACE sample, builds options, determines the gold answer, runs Model A to score original options, calls Model B to generate distractors and hints, and returns a unified quiz item.

```

1 quiz_item = {
2     "article": article,
3     "question": question,
4     "original_options": options,
5     "correct_label": correct_label,
6     "correct_answer": correct_answer,

```

```

7     "model_a_prediction": predicted_label,
8     "model_a_confidence": confidence,
9     "model_a_scores": model_a_scores,
10    "generated_distractors": distractors,
11    "generated_hints": hints,
12 }

```

Listing 8: Simplified quiz item construction.

## 9.2 Required UI Screens

The project required four UI screens/views. The implemented Streamlit interface includes all four. The UI is designed for rapid ML demonstration without a separate frontend stack.

Table 21: UI requirement coverage.

| Required screen     | Implemented behavior                                                                            | Status   |
|---------------------|-------------------------------------------------------------------------------------------------|----------|
| Article Input       | Load a selected or random RACE sample; optional custom article mode; toggle Model B distractors | Complete |
| Quiz                | Display question, options, check answer, show correctness and Model A prediction                | Complete |
| Hint Panel          | Reveal progressive hints one at a time, then reveal answer                                      | Complete |
| Developer Dashboard | Model confidence, session history, inference latency, Model B output, charts, CSV export        | Complete |

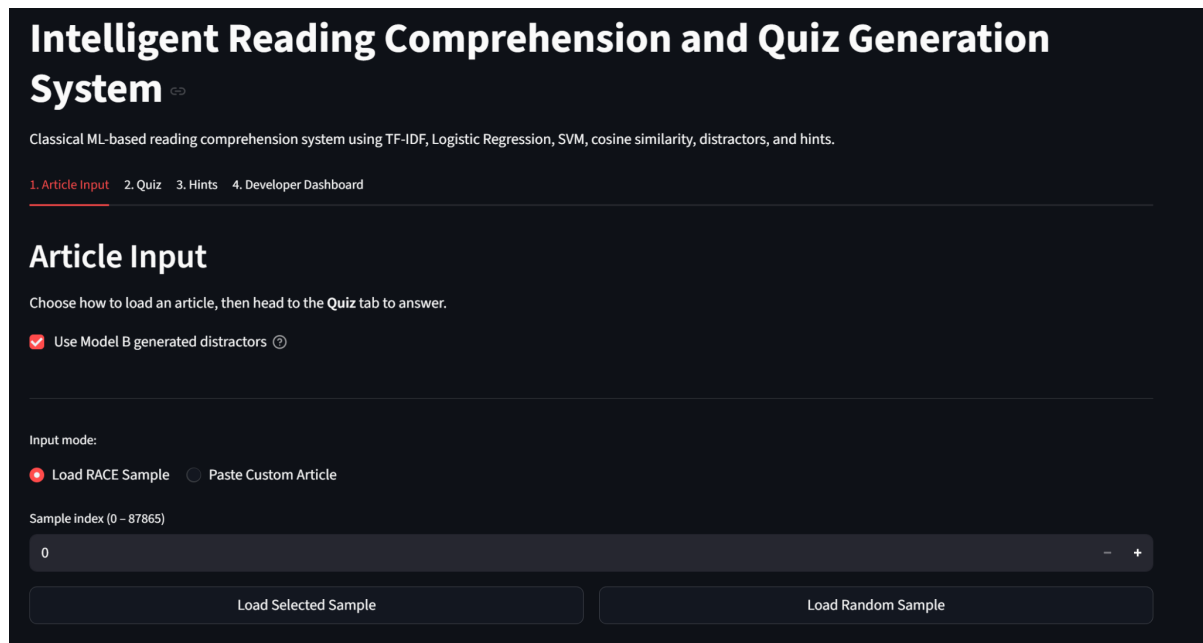


Figure 10: Streamlit UI - Article Input tab before loading a sample.



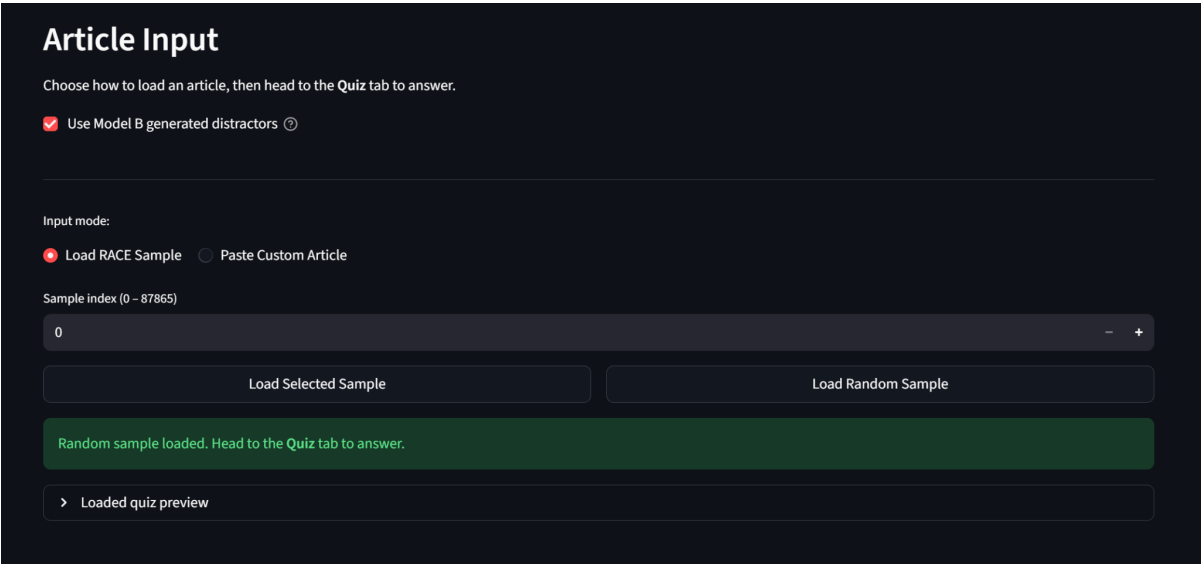


Figure 11: Streamlit UI - Article Input tab after loading a random RACE sample.

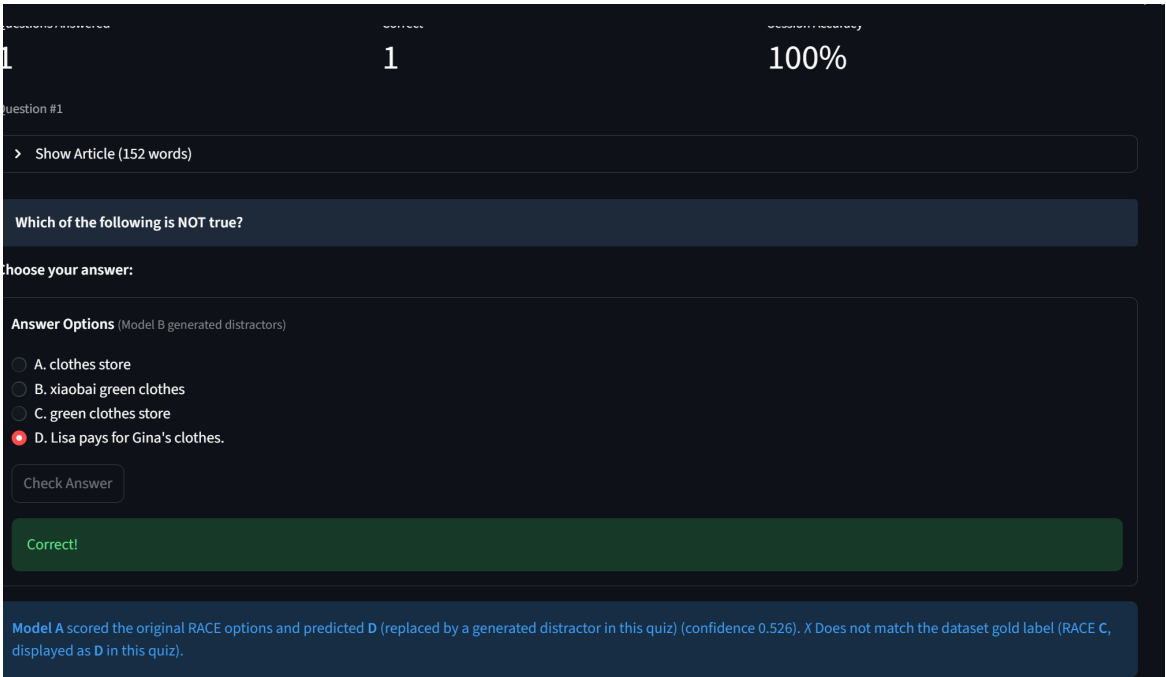


Figure 12: Streamlit UI - Quiz tab with generated distractors and a checked answer.

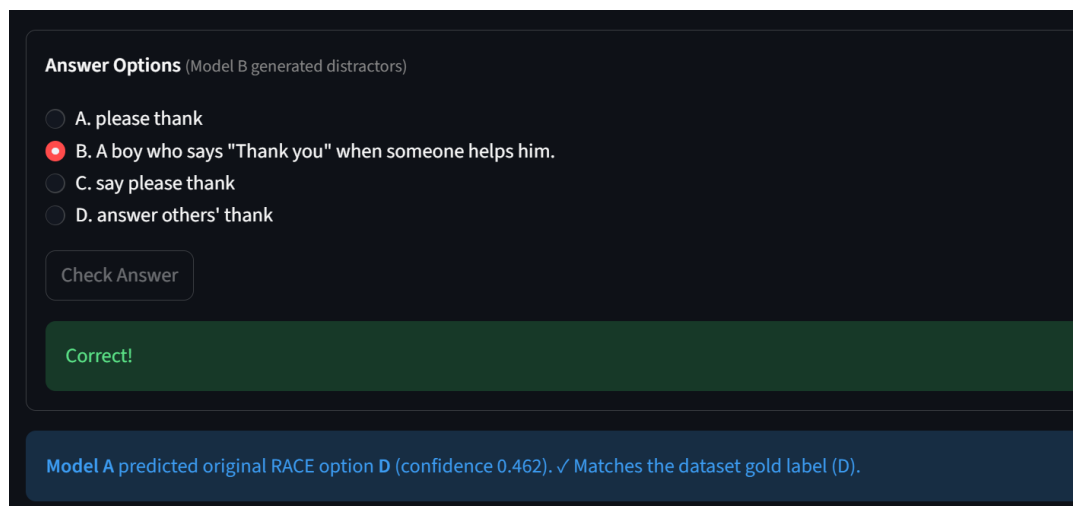


Figure 13: Streamlit UI - Answer checking and Model A feedback message.

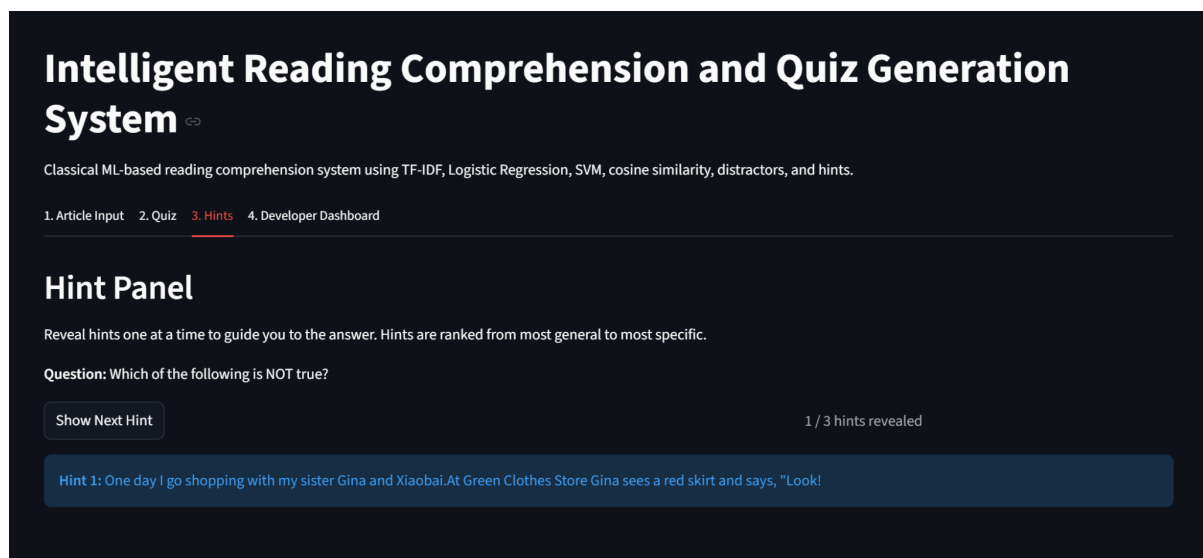


Figure 14: Streamlit UI - Hint Panel showing progressive hint reveal.

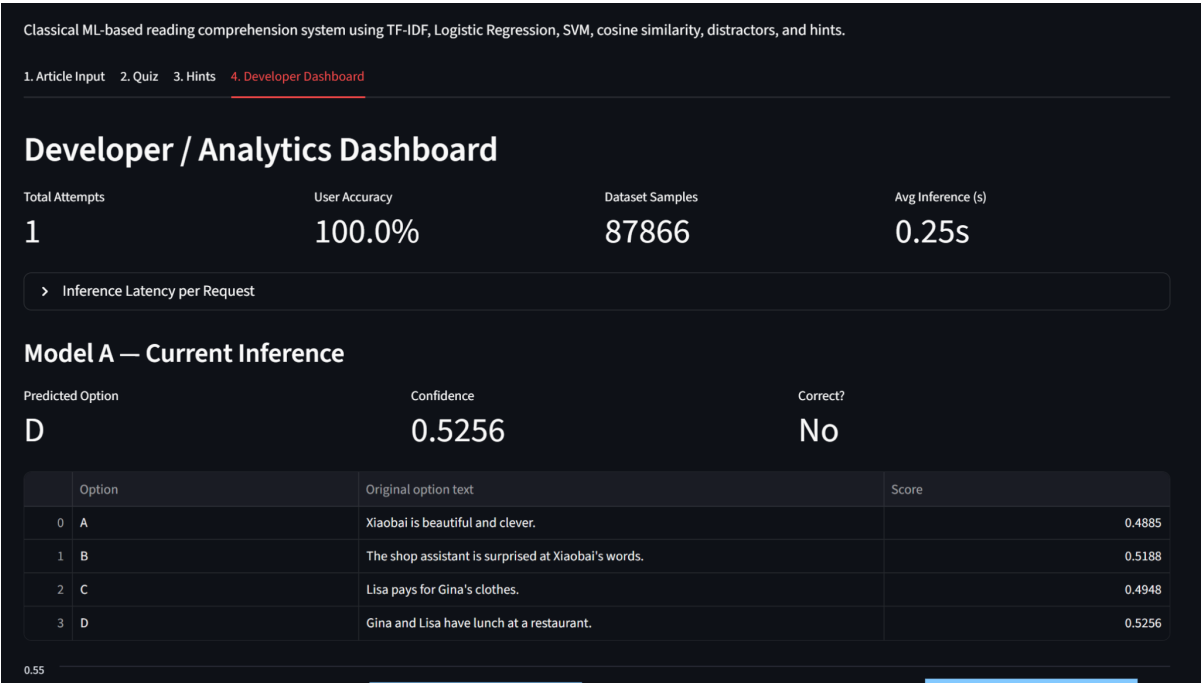


Figure 15: Streamlit UI - Developer Dashboard overview with inference latency and current model output.

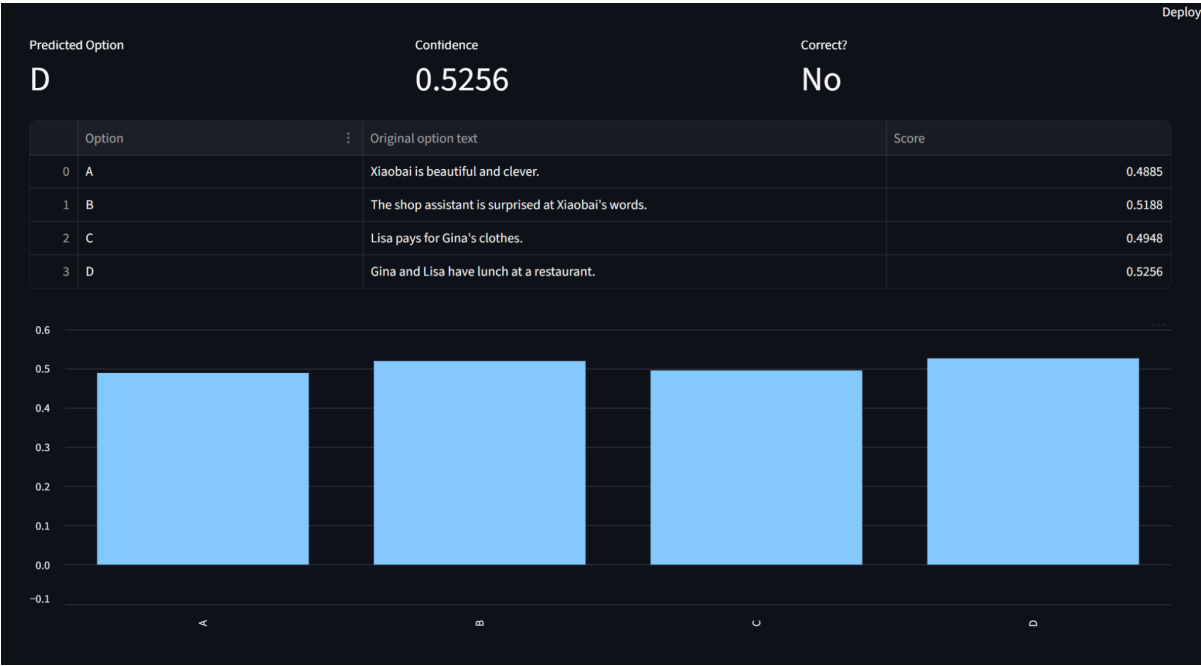


Figure 16: Streamlit UI - Developer Dashboard chart of Model A option scores.

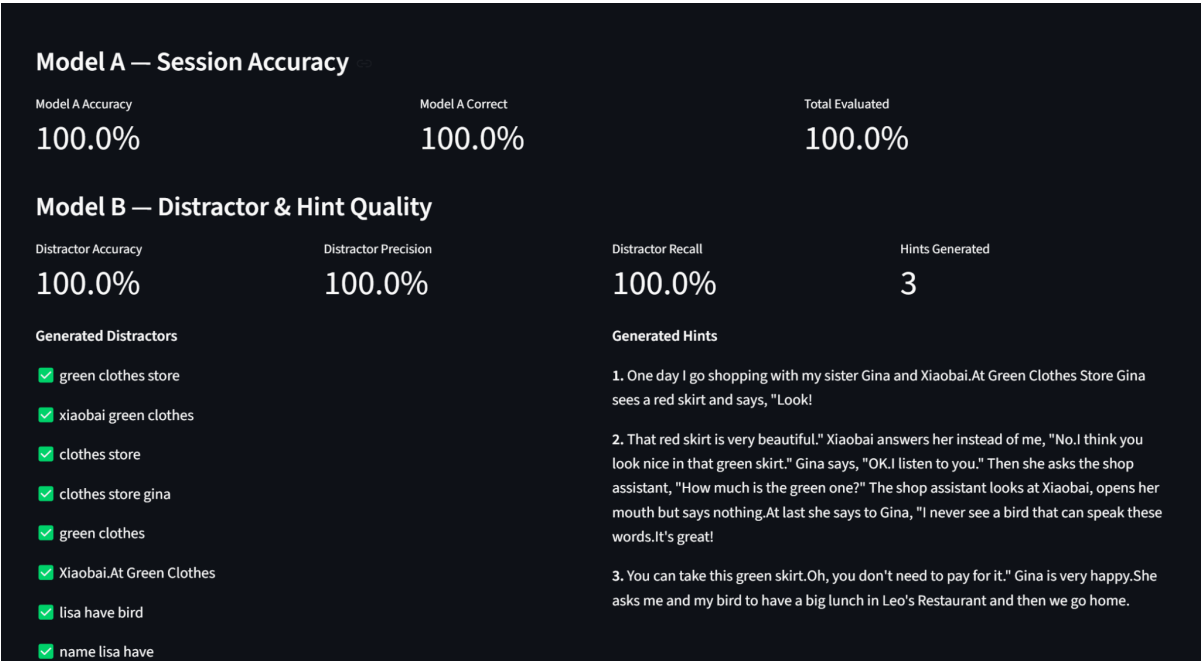


Figure 17: Streamlit UI - Model B distractor and hint quality section.

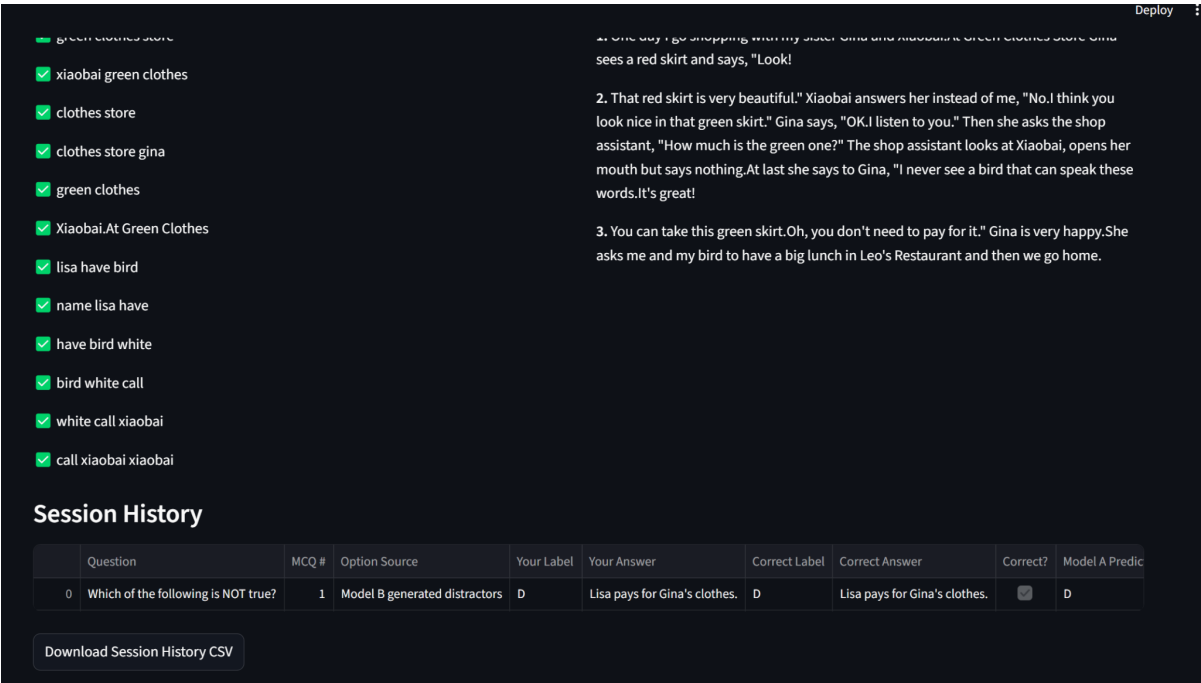


Figure 18: Streamlit UI - Session history and CSV download functionality.

9.3 Session State and Usability

Streamlit reruns scripts whenever users interact with widgets, so session state is essential. The app stores the current quiz item, selected answer, checked status, hint count, session history, and latency statistics.

```
1 if "quiz_item" not in st.session_state:
2     st.session_state.quiz_item = None
3 if "history" not in st.session_state:
```

```

4     st.session_state.history = []
5     if "hint_count" not in st.session_state:
6         st.session_state.hint_count = 0

```

Listing 9: Typical Streamlit session-state variables used by the app.

## 10 Results and Evaluation

### 10.1 Evaluation Metrics

The project evaluates classification, generation, and UI interaction separately. Classification metrics apply to Model A. Text generation metrics apply to template-based QG. Distractor/hint evaluation is partly automatic and partly qualitative.

Table 22: Evaluation metrics used by component.

| Component                   | Metrics / outputs                                                                    |
|-----------------------------|--------------------------------------------------------------------------------------|
| Model A answer verification | Accuracy, macro F1, precision, recall, ROC-AUC, confusion matrix, exact match        |
| Model A template QG         | BLEU, ROUGE-L, METEOR support, qualitative question examples                         |
| Model A clustering          | KMeans silhouette score                                                              |
| Model B distractors         | Validity tests, precision/recall/F1 utility, qualitative examples, generated options |
| Model B hints               | Hint count, non-empty string tests, qualitative relevance                            |
| UI                          | User accuracy, session history, inference latency, dashboard charts, CSV export      |

### 10.2 Model A Result Interpretation

The Linear SVM is selected as the strongest classical supervised model based on macro F1 and positive-class recall. Logistic Regression remains useful because it gives probability outputs and ROC-AUC. The result gap between LR and SVM is consistent with common NLP practice: linear SVM often performs well on sparse TF-IDF features.

Table 23: Detailed class-wise Model A report.

| Model               | Class       | Precision | Recall | F1   | Support |
|---------------------|-------------|-----------|--------|------|---------|
| Logistic Regression | 0 incorrect | 0.78      | 0.60   | 0.68 | 3000    |
| Logistic Regression | 1 correct   | 0.29      | 0.49   | 0.37 | 1000    |
| Linear SVM          | 0 incorrect | 0.81      | 0.62   | 0.70 | 3000    |
| Linear SVM          | 1 correct   | 0.33      | 0.57   | 0.42 | 1000    |

### 10.3 Question Generation Result Interpretation

The BLEU and ROUGE-L scores are low, which is expected for a traditional template system. RACE questions are often paraphrased, inferential, or phrased as global questions such as “Which of the following is true?” A template system may generate a grammatically valid question that tests

a similar fact but does not match the reference wording. Therefore, overlap metrics underestimate usefulness in some cases.

Table 24: Question generation output analysis.

| Observation                           | Explanation                                                                          |
|---------------------------------------|--------------------------------------------------------------------------------------|
| BLEU is very low                      | Template questions rarely match exact n-gram wording of human-written RACE questions |
| ROUGE-L is higher than BLEU           | Generated questions share some longer lexical sequences with references              |
| METEOR support is useful              | METEOR can reward stemming and synonym matches when NLTK resources are available     |
| Generated questions are interpretable | Each output has a source sentence, answer span, and template label                   |

## 10.4 Model B Result Interpretation

Model B produces usable distractor/hint outputs and is integrated into the UI. In some examples, generated distractors are still short phrases or extracted surface spans. This is a limitation of classical candidate extraction. However, the generated options are transparent and easy to inspect, and the dashboard makes distractor quality visible.

## 10.5 Unit Test Results

The project passed all 40 recorded unit tests, covering evaluation utilities, Model A inference, Model B mocked functions, and Model B real inference.

Table 25: Unit test completion summary.

| Test suite                        | Tests run | Runtime   | Status |
|-----------------------------------|-----------|-----------|--------|
| <code>tests.test_inference</code> | 40        | 5.632 sec | OK     |

## 10.6 External Benchmark Comparison

The project compares its classical approach against published neural/transformer work. Because the tasks differ slightly, the comparison is split into answer verification and distractor generation rather than forcing all scores into one metric.

Table 26: Classical implementation versus external neural benchmark context.

| System                 | Implemented here? | Score type          | Reported result / interpretation                          |
|------------------------|-------------------|---------------------|-----------------------------------------------------------|
| Logistic Regression    | Yes               | Accuracy / Macro F1 | 0.5745 / 0.5230 on validation sample                      |
| Linear SVM             | Yes               | Accuracy / Macro F1 | 0.6055 / 0.5608; best implemented supervised model        |
| Template QG            | Yes               | BLEU / ROUGE-L      | 0.016998 / 0.132267; low but expected for rule-based QG   |
| Baseline T5 DG         | No                | RACE-DG BLEU avg    | BLEU1 0.20, BLEU4 0.07 in published benchmark             |
| Two-step T5 DG         | No                | RACE-DG BLEU avg    | BLEU1 0.29, BLEU4 0.10 in published benchmark             |
| Human RACE performance | No                | MCQA accuracy       | Original RACE paper reports high human ceiling around 95% |

## 11 Discussion, Limitations, Ethics, and Future Work

### 11.1 Discussion

The final system demonstrates a complete classical ML approach to reading comprehension and quiz generation. The most important technical lesson is that accuracy alone is not reliable under option-level class imbalance. The early 75% accuracy model was invalid because it predicted all samples as incorrect. After class weighting, the models became less accurate numerically but much more meaningful because they predicted both classes and improved positive-class recall.

The best model, Linear SVM, achieved macro F1 0.5608. This should be interpreted in context: RACE is a difficult reasoning dataset, and the project intentionally avoids neural contextual embeddings. A classical TF-IDF model cannot deeply understand paraphrase, negation, common-sense, or multi-sentence reasoning. Its value lies in interpretability, speed, and compliance with the traditional ML requirement.

## 11.2 Limitations

- **Limited semantic understanding:** TF-IDF treats words lexically and does not understand synonyms or paraphrases.
- **Question generation quality:** Template-based QG produces grammatical questions for simple factual sentences but struggles with complex reasoning questions.
- **Distractor quality:** Classical candidate extraction can produce short phrases or surface-level distractors.
- **Metric limitations:** BLEU and ROUGE reward lexical overlap, which may not fully reflect educational usefulness.
- **Model A confidence:** Option scores are often close, indicating uncertainty and limited separation.
- **Clustering weakness:** KMeans silhouette is low because sparse text vectors do not form clean clusters.

## 11.3 Ethical Considerations

The system should be treated as an educational aid, not a replacement for human exam authors. Generated questions and distractors may be wrong, biased, misleading, or too easy. The RACE dataset is drawn from Chinese school examinations and may contain cultural, linguistic, or educational biases. The UI should make it clear that AI-generated outputs may contain errors and should be reviewed by instructors before use in real assessment settings.

## 11.4 Future Work

Future improvements should focus on higher-quality distractor generation, richer question generation, and stronger evaluation.

1. Add POS tagging or dependency parsing to improve question templates.
2. Use WordNet or distributional thesauri to generate more grammatical distractors.
3. Add human evaluation forms for distractor plausibility.
4. Add stratified 80/10/10 splitting by answer label.
5. Add METEOR to the standard evaluation output and store it in the final CSV.
6. Improve multi-question flow so one article can generate several quiz questions.
7. Add side-by-side mode: original RACE question vs generated template question.
8. Add more robust result logging and experiment tracking.



## 12 Conclusion and Rubric Mapping

### 12.1 Conclusion

This project successfully implements an end-to-end Intelligent Reading Comprehension and Quiz Generation System using classical machine learning. It includes a reproducible data pipeline, option-level answer verification, TF-IDF and cosine similarity features, Logistic Regression, Linear SVM, unsupervised clustering, template-based question generation, distractor generation, hint generation, a unified quiz pipeline, a Streamlit interface, evaluation scripts, and unit tests. The best implemented supervised model is Linear SVM, with accuracy 0.6055 and macro F1 0.5608. The UI is functional and includes all required views. The report also compares the classical implementation with published transformer-based distractor-generation benchmarks on RACE-DG.

The system's main strength is that it is fully aligned with the traditional ML requirement while still demonstrating meaningful NLP, evaluation, and software engineering. Its main limitation is that classical lexical representations cannot match the semantic reasoning power of transformer models. Nevertheless, the implementation is transparent, reproducible, and pedagogically suitable for the course project.

### 12.2 Rubric Mapping

Table 27: Rubric coverage summary.

| Rubric component                     | Marks | Status   | Evidence in implementation/report                                            |
|--------------------------------------|-------|----------|------------------------------------------------------------------------------|
| EDA and preprocessing                | 10    | Complete | 80/10/10 split, schema, option-level expansion, class-balance analysis       |
| Model A traditional ML               | 15    | Complete | Logistic Regression, Linear SVM, TF-IDF, cosine features, metrics            |
| Model A unsupervised/semi-supervised | 20    | Complete | KMeans clustering with silhouette score; repo also references LP/GMM support |
| Model A ensemble                     | 5     | Complete | Inference combines LR probability and SVM score                              |
| Model B distractor generation        | 15    | Complete | Generated distractors, Model B dashboard, tests, benchmark comparison        |
| Model B hint generation              | 10    | Complete | Progressive hints with UI reveal and sentence ranking                        |
| User interface                       | 15    | Complete | Four Streamlit tabs and screenshots                                          |
| Final report                         | 5     | Complete | Research-style report with figures, tables, references, limitations          |
| Code quality                         | 5     | Complete | Modular files, tests, joblib artifacts, reproducibility commands             |

## 12.3 Reproducibility Commands

```
1 # 0. Split the Kaggle file into 80/10/10
2 python src/split_dataset.py --source data/raw/race.csv
3
4 # 1. Preprocess option-level data
5 python src/preprocessing.py
6
7 # 2. Train Model A
8 python src/model_a_train.py
9
10 # 3. Train Model B
11 python src/model_b_train.py
12
13 # 4. Evaluate template question generation
14 python src/question_generation_eval.py --sample-size 20 --top-k 3
15
16 # 5. Run tests
17 python -m unittest tests.test_inference -v
18
19 # 6. Launch UI
20 streamlit run ui/app.py
```

Listing 10: Recommended final command sequence.

## References

- [1] Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. 2017. *RACE: Large-scale ReAding Comprehension Dataset From Examinations*. In Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing, pages 785–794, Copenhagen, Denmark. Association for Computational Linguistics. DOI: 10.18653/v1/D17-1082.
- [2] Shiva Taslimipoor, Luca Benedetto, Mariano Felice, and Paula Buttery. 2024. *Distractor Generation Using Generative and Discriminative Capabilities of Transformer-based Models*. In LREC-COLING 2024, pages 5052–5063.
- [3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. In Proceedings of NAACL-HLT 2019.
- [4] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. Journal of Machine Learning Research, 21(140):1–67.
- [5] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. *BLEU: a Method for Automatic Evaluation of Machine Translation*. In Proceedings of ACL 2002.
- [6] Chin-Yew Lin. 2004. *ROUGE: A Package for Automatic Evaluation of Summaries*. In Text Summarization Branches Out: Proceedings of the ACL-04 Workshop.
- [7] Satanjeev Banerjee and Alon Lavie. 2005. *METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments*. In Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization.
- [8] Yifan Gao, Lidong Bing, Piji Li, Irwin King, and Michael R. Lyu. 2019. *Generating Distractors for Reading Comprehension Questions from Real Examinations*. In AAAI 2019.
- [9] Zhaopeng Qiu, Xian Wu, and Wei Fan. 2020. *Automatic Distractor Generation for Multiple Choice Questions in Standard Tests*. In Proceedings of COLING 2020.
- [10] Semere Kiros Bitew, Johannes Deleu, Chris Develder, and Thomas Demeester. 2023. *Distractor Generation for Multiple-choice Questions with Predictive Prompting and Large Language Models*. arXiv preprint arXiv:2307.16338.
- [11] Fabian Pedregosa et al. 2011. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12:2825–2830.
- [12] Streamlit. 2024. *Streamlit Documentation*. Available at <https://docs.streamlit.io/>. Accessed May 2026.
- [13] Ankit Dhiman. *RACE Dataset*. Kaggle dataset package. Available at <https://www.kaggle.com/datasets/ankitdhiman7/race-dataset>. Accessed May 2026.